Formula 1 Podium Prediction

CSEN903 Advanced Computer Lab | Milestone 1 Report | ACL_Team70

Team members

#	Name	Student ID	Tutorial
1	Ahmed Mohamed Abdelkhalek	58-3594	T05
2	Yousef Shawky	58-8277	T11
3	Amr Amin Koura	58-3669	T28
4	Ahmed Yasser Elsawi	58-2053	T08

Summary

We built a clean table with one row per driver per race and used it to predict whether a driver finishes on the podium (top 3). We only use information that is known after qualifying and before the race starts. We also answered three data questions about the races.

- **Data:** 14 tables cleaned (the text `\N` turned into real missing values, dates and lap times turned into real types, 91 repeated result rows removed). The modeling table has 8,970 rows (2003 to 2024), 14.3% of them podiums.
- **Features:** 12 features in 5 groups: grid, qualifying, standings before the race, recent form, and circuit history. None of them uses information from the race itself.
- **Models:** a simple grid rule, logistic regression (baseline), two random forests and a shallow neural network. All models beat the grid rule.
- **Final model:** random forest B (shallow trees), chosen on the validation PR-AUC (0.744). On the unseen test years 2022 to 2024 it reaches PR-AUC 0.702, ROC-AUC 0.936 and F1 0.664.
- **Most useful information:** the starting position (grid and qualifying), then recent team and driver form and the championship standings. XAI shows what the model relies on, not what causes a podium.

1. Task and prediction cutoff

Unit of analysis: one driver in one race, so the pair (raceId, driverId) must be unique. **Target:** podium = 1 if the driver's final position (positionOrder) is 3 or better, else 0.

Cutoff: we predict after qualifying and before the race starts. So we may use the grid, qualifying, the standings after the previous race and results of earlier races. We may **not** use anything from this race: finishing position, status, points, laps, pit stops, lap times, or the standings after this race. Using them would be **data leakage**: the model would see the answer, and its scores would look much better than they really are.

2. Data audit and cleaning

The dataset has 14 linked tables. We never change the raw CSV files; all cleaning is done on copies in pandas.

2.1 First look at the raw data

Many columns use the text `\N` for "missing". Because it is text, pandas does not count it as missing, so the real number of missing values is hidden. We counted it for every column of every table (Figure 1). It appears in 6 of the 14 tables, mostly in columns that only exist for some rows, for example race times of drivers who did not finish, Q2/Q3 times of drivers knocked out earlier, and session dates of older races.

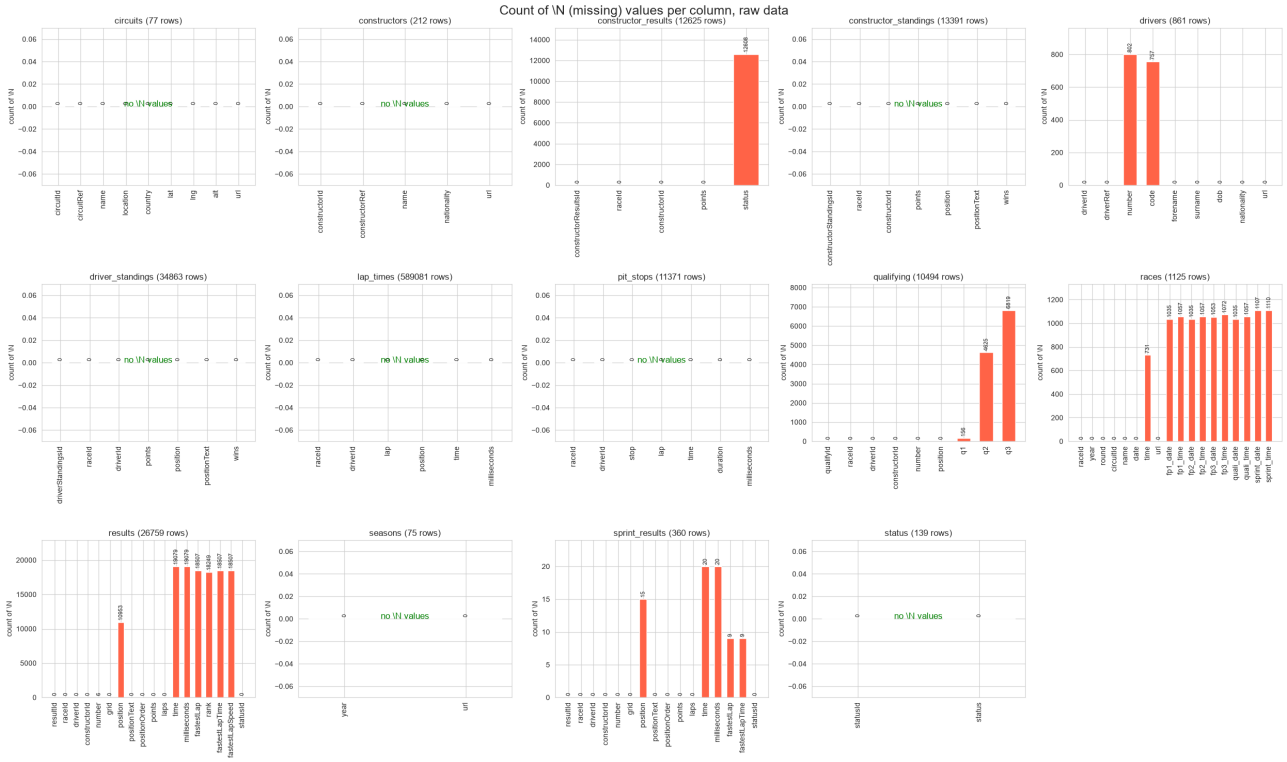


Figure 1: count of the text \N per column in every raw table.

2.2 Repeated driver-race rows

The results table has 85 (raceId, driverId) pairs that appear more than once (176 rows, 91 extra). Almost all are from the 1950s, when drivers shared cars during a race (Figure 2). **Rule:** for each pair we keep the row with the best (lowest) positionOrder. A driver's result in a race is the best result among the cars they drove, and that row decides the podium target. The rule is simple and gives the same result every time.

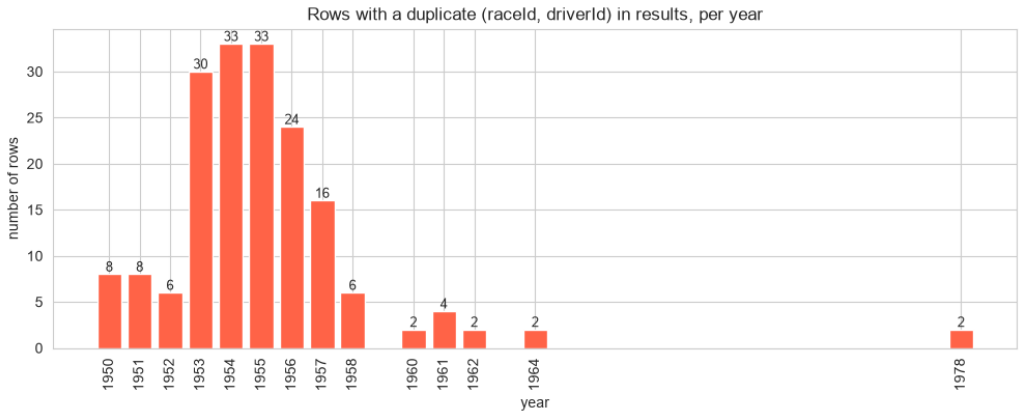


Figure 2: rows with a repeated (raceId, driverId) pair, per year.

2.3 Keys, sentinels and types

- **Primary keys:** unique and never empty in every table. **Foreign keys:** every id points to a row that exists in its parent table (0 problems in all checks), so the tables can be joined safely.
- **Sentinels:** every `\N` was replaced by a real missing value (NaN). After cleaning, 0 are left.
- **Types:** 24 columns were converted: dates to real dates, lap and qualifying times like "1:26.572" to seconds (86.572), and number columns stored as text to numbers.

2.4 Findings after cleaning that shaped the model

The target is imbalanced: only 12.7% of all rows are podiums (Figure 3). A model that always says "no podium" would be 87% accurate and useless, so we do not use accuracy (see section 6).

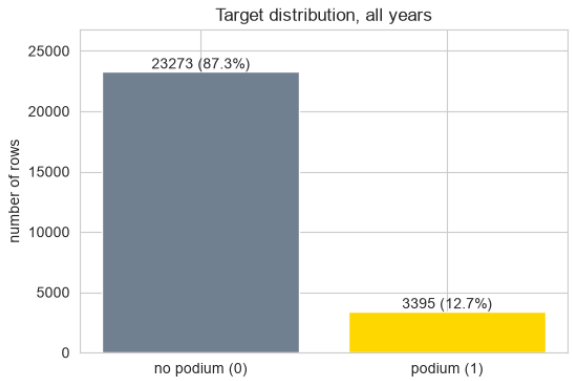


Figure 3: target distribution over all years.

Qualifying coverage drift: qualifying data does not exist before 1994, is patchy from 1996 to 2002 and is about 100% complete from 2003 (Figure 4). Field sizes are also stable (about 20 to 24 cars) from 2003. So the model uses **2003 to 2024** only.

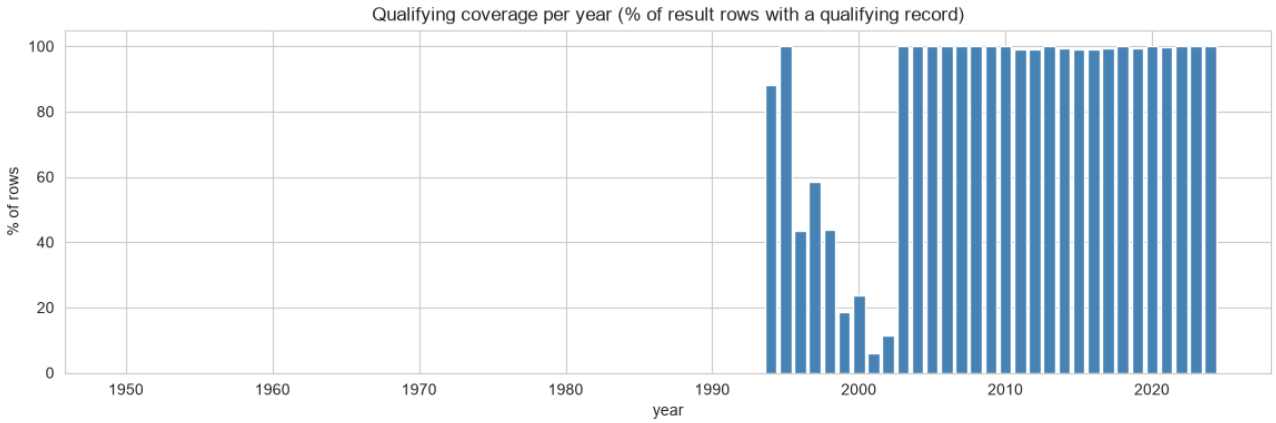


Figure 4: share of result rows with a qualifying record, per year.

3. Data-engineering questions

All three answers use the joined table de: results joined with races, circuits, qualifying, drivers, constructors and status. All joins are left joins, and the row count stays the same (one row = one driver in one race).

Q1. Which circuits turn a front-row start into a podium most often, before and after 2014?

Method: 2003 to 2013 vs 2014 to 2024 (11 seasons each, full qualifying data). Rate = podiums / front-row starts. We use two definitions of front row: grid position 1 or 2 (where the driver actually started) and qualifying position 1 or 2. They differ because of grid penalties. Only circuits raced in both eras with at least 10 front-row starts in each are compared (14 circuits).

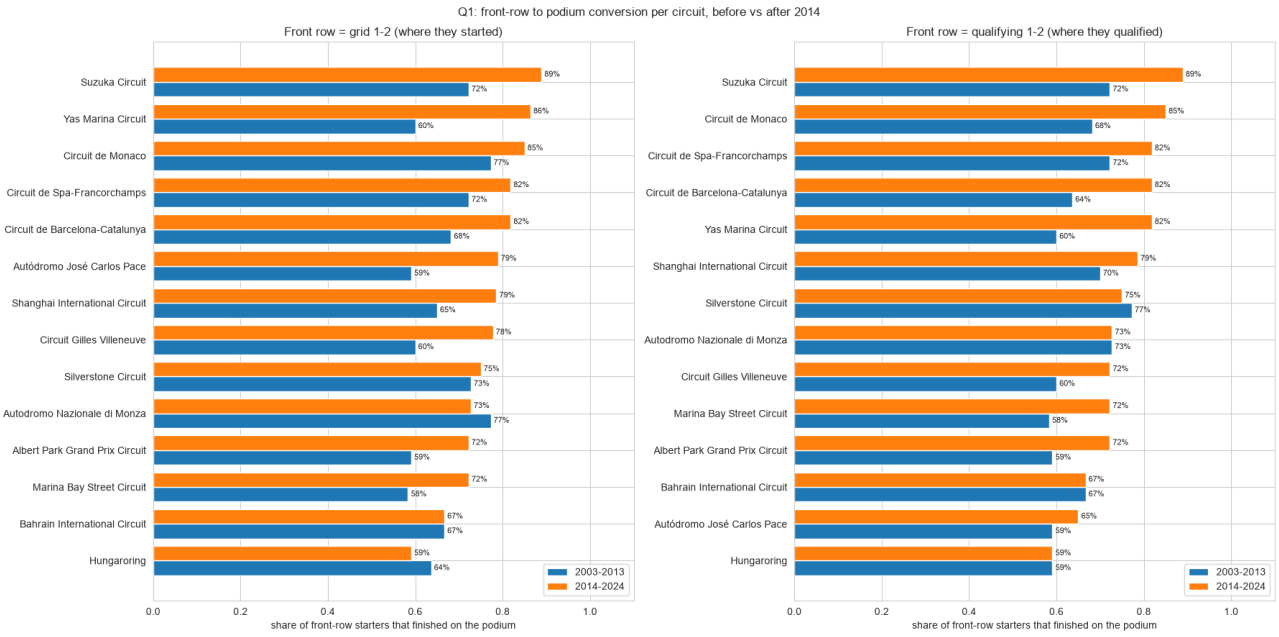


Figure 5: front-row to podium rate per circuit, grid definition (left) and qualifying definition (right).

Answer: since 2014 the top circuits are Suzuka (89%), Yas Marina (86%) and Monaco (85%). Before 2014 Monaco and Monza led (77%), followed by Silverstone, Suzuka and Spa (72% to 73%). The ranking changed: Monaco is the only circuit near the top in both eras. Yas Marina (60% to 86%) and Interlagos (59% to 79%) rose the most, while Monza and the Hungaroring are the only circuits that dropped. Overall, a front-row start became more valuable in the hybrid era (68% to 76%).

Qualifying vs grid: the two definitions disagree in about 4% of front-row cases. Where penalised front-row qualifiers were common, the qualifying-based rate is clearly lower: Monaco before 2014 (77% grid vs 68% qualifying) and Interlagos after 2014 (79% vs 65%).

Grain, missingness, cutoff: one row = one driver in one race, 2003 to 2024 (complete qualifying data). With about 20 starts per circuit and era, one podium changes a rate by about 5 points, so small differences should not be over-interpreted.

Q2. Do drivers podium more often at home, when we compare drivers from similar grid positions?

Method: a race is a home race when the driver's nationality matches the circuit's country (for example British to UK). To control for starting position, we group starts into grid buckets (P1-3, P4-6, P7-10, P11-15, P16+ and pit lane) and compare home and away podium rates **inside each bucket**.

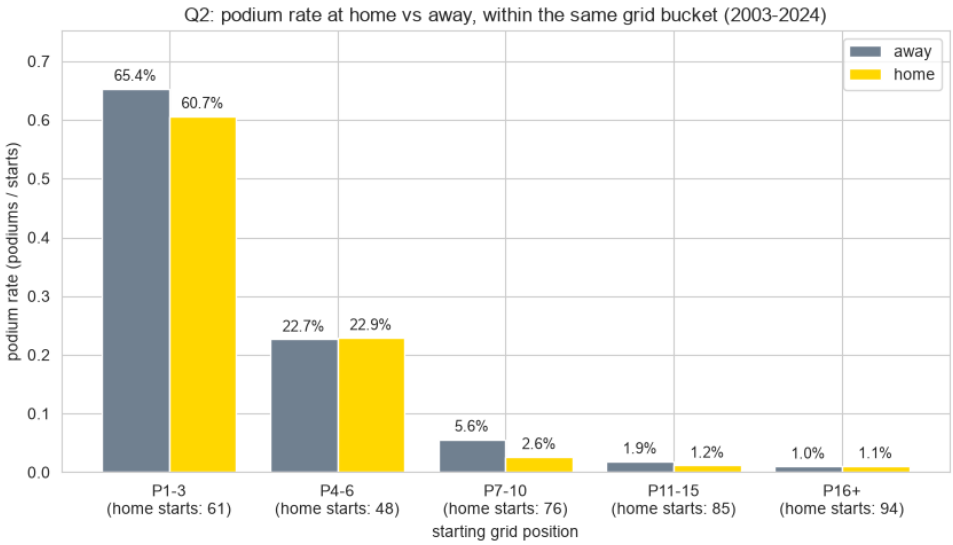


Figure 6: podium rate at home vs away inside each grid bucket (2003 to 2024).

Answer: no, there is no home advantage. Inside every bucket the rates are almost the same (P1-3: 61% home vs 65% away, P4-6: 23% vs 23%). If home drivers had done as well as away drivers from the same buckets, they would have reached about 57.5 podiums; they actually reached 52. Without the grid control both groups look identical (14.2% vs 14.3%). The gap of about 5 podiums over 22 seasons is too small to be a real effect, so the answer is "no evidence of a home advantage", not "home drivers do worse".

Grain, missingness, cutoff: one row = one driver in one race, 2003 to 2024. Only 365 of 8,970 starts are home starts, because 12 nationalities (for example Finnish, Danish, Swiss) have no circuit in their country. The pit-lane bucket has only 1 home start and is not compared. Home country is a proxy based on nationality.

Q3. Which constructors have the highest mechanical-retirement rate in 2014-2021 vs 2022-2024?

Method: rate = mechanical retirements / starts, per constructor and era. A retirement counts as mechanical when a part of the car failed by itself: 37 statuses such as Engine, Power Unit, Gearbox, Brakes and Hydraulics. Not counted: finishes (+N Laps), crashes (Collision, Accident, Spun off), driver or rule reasons (Disqualified, Withdrew, Illness) and unclear statuses (Retired, Puncture, Tyre, Wheel, wings), which can also come from contact or debris. Teams need at least 40 starts in an era.

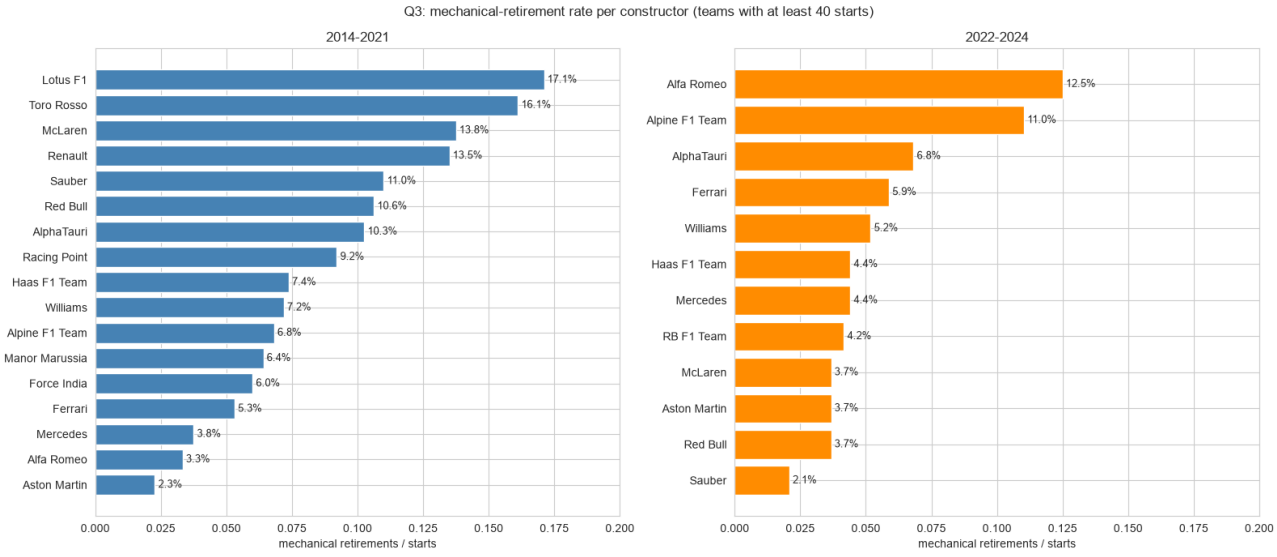


Figure 7: mechanical-retirement rate per constructor, 2014-2021 (left) and 2022-2024 (right), same scale.

Answer: in 2014-2021 the highest rates were Lotus F1 (17%), Toro Rosso (16%), McLaren (14%) and Renault (14%); Mercedes was the most reliable (3.8%). In 2022-2024 the highest were Alfa Romeo (12.5%) and Alpine (11%); all other teams are at 7% or below, and Red Bull, McLaren and Aston Martin share the lowest rate

(3.7%). Overall the cars became more reliable (9.2% to 5.7% of starts). McLaren and Red Bull improved the most, while Alpine (the renamed Renault team) stayed near the top.

Grain, missingness, cutoff: one row = one driver in one race, 2014 to 2024 (4,626 starts). Team names are used as they are in the data, so rebrands count as separate teams (for example Toro Rosso, AlphaTauri, RB). With about 136 starts per team in 2022-2024, one failure changes a rate by about 0.7 points. The status is only known after the race, so this is an analysis question and never a model feature.

4. Modeling table and features

The modeling table starts from the cleaned results (repeated pairs already removed) and joins races and qualifying. It has **8,970 rows** for 2003 to 2024, **0 repeated (raceId, driverId) pairs**, and 14.3% podiums. Form features are computed on all years first, so that a driver's form in early 2003 can use their 2002 races; the table is then cut to 2003 to 2024.

4.1 Selected features and why each is valid before the race

Group	Feature	What it is	Why it is known before the race
Grid	grid	starting position (pit-lane starts moved to the last slot)	set before the start
Grid	pit_lane_start	1 if the driver starts from the pit lane (grid = 0 in the data)	set before the start
Grid	grid_penalty	grid minus qualifying position (places lost to penalties)	both known before the start
Qualifying	quali_position	qualifying result	qualifying is finished
Qualifying	gap_to_pole_pct	best qualifying lap vs pole lap, in %	qualifying is finished
Standings	driver / constructor points and rank before	championship points and rank after the **previous** round	previous round only (round + 1 join)
Form	driver_podium_rate_prev5	driver's podium share in their previous 5 races	shift(1): earlier races only
Form	constructor_podium_rate_prev5	team's podium rate over its previous 5 races	shift(1): earlier races only
Circuit history	driver_circuit_podium_rate	driver's podium share in earlier races at this circuit	shift(1): earlier races only

How we avoid leakage: all form features use `shift(1)` within each driver or team, so the current race is never part of its own feature. The standings table stores points *after* each race, so we add 1 to the round number and join: the standings after round N become the features of round N+1.

Choices we made: grid = 0 means a pit-lane start, which is behind every car; as a plain number it would look better than pole, so we flag it and move it to the back. Raw Q1/Q2/Q3 lap times cannot be compared across circuits (Monaco is about 70 s, Spa about 105 s), so we use the gap to pole in %. We left out driver, team and circuit IDs (names, not measurements; new drivers and teams in the test years have no learned value), the year (the test years are later than all training years), and a home-race flag (Q2 found no home advantage).

4.2 Evidence for the features

Before training, we checked that every feature moves with the podium rate (Figures 8 and 9).

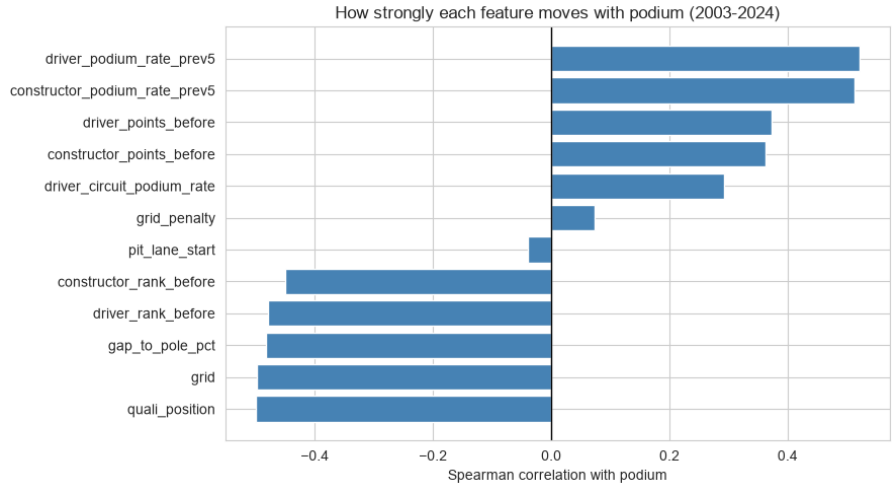


Figure 8: Spearman correlation of every feature with podium (2003 to 2024).

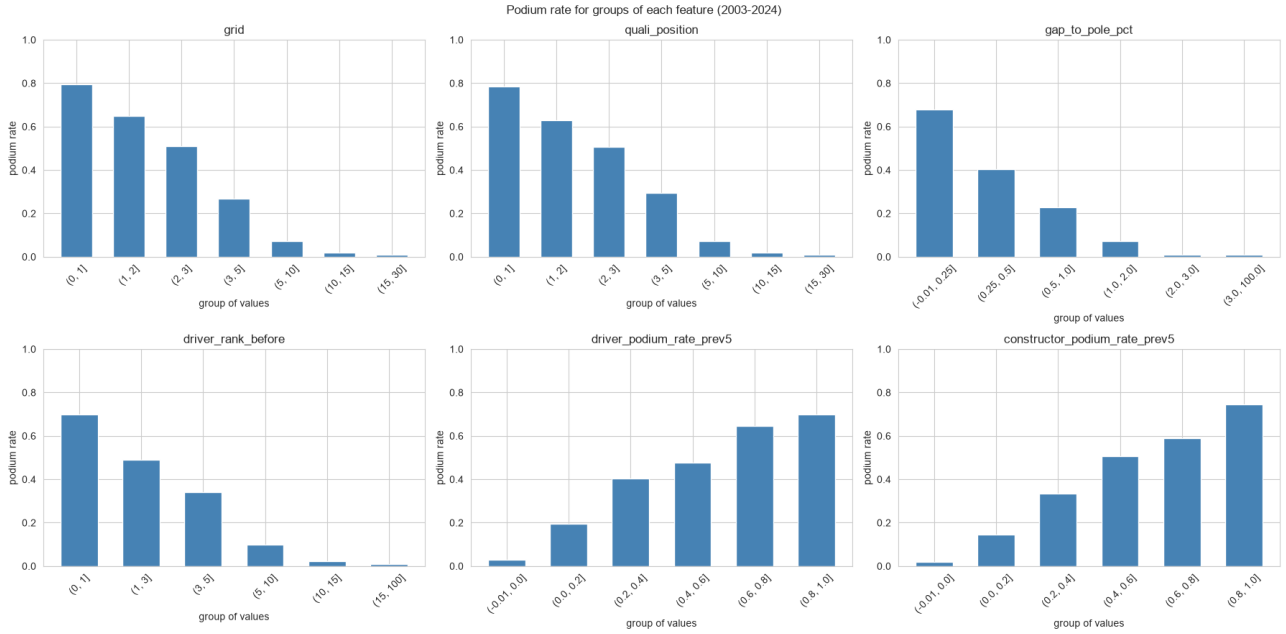


Figure 9: podium rate for groups of values of the main features. A group like (1, 3] means above 1 and up to 3.

- **Grid and qualifying** are the strongest signals: starting P1 becomes a podium about 79% of the time, P6 to P10 about 7%, P11 or worse almost never. Gap to pole shows the same (within 0.25% of pole: about 68%).
- **Standings:** the championship leader podiums about 70% of the time, drivers ranked 6 to 10 about 10%.
- **Form:** the podium rate rises steadily with driver and team form, from about 2% (no recent podium) to about 70%. These have the highest correlations (+0.52 and +0.51).
- **Weaker features:** circuit history (+0.29), grid penalty (+0.07) and pit-lane start (-0.04). We keep them because they add information the others do not have; the ablation (section 7.1) tests each group.

5. Split and preprocessing

We split by **season**, not at random, because the real use is to predict future races from past ones. **Train:** 2003 to 2018 (6,411 rows). **Validation:** 2019 to 2021 (1,200 rows), used to choose models, thresholds and settings. **Test:** 2022 to 2024 (1,359 rows), unseen and used only once at the end. Every race stays in one set, and the podium rate is about 14% to 15% in all three (Figure 10). The test years also start new car rules (2022), so the test score shows how well the model handles a new era.

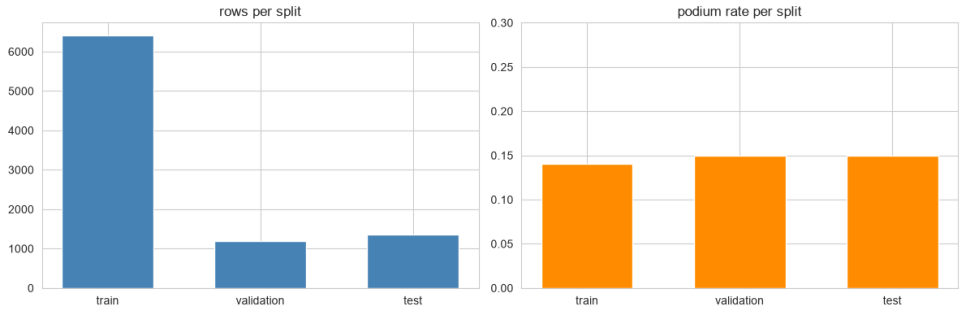


Figure 10: rows and podium rate per split.

Preprocessing (learned on the training rows only, then applied to validation and test):

- **Fill missing values with the training median.** Missing values come from a driver's first race (no form yet), round 1 (no standings yet) and missing qualifying times. The median is not pulled by very large values.
- **Scale** every feature to mean 0 and standard deviation 1. Logistic regression and the neural network need this (points go up to 722, rates are between 0 and 1). The random forest does not need scaling, so it gets the filled but unscaled data, and its explanations show real values.

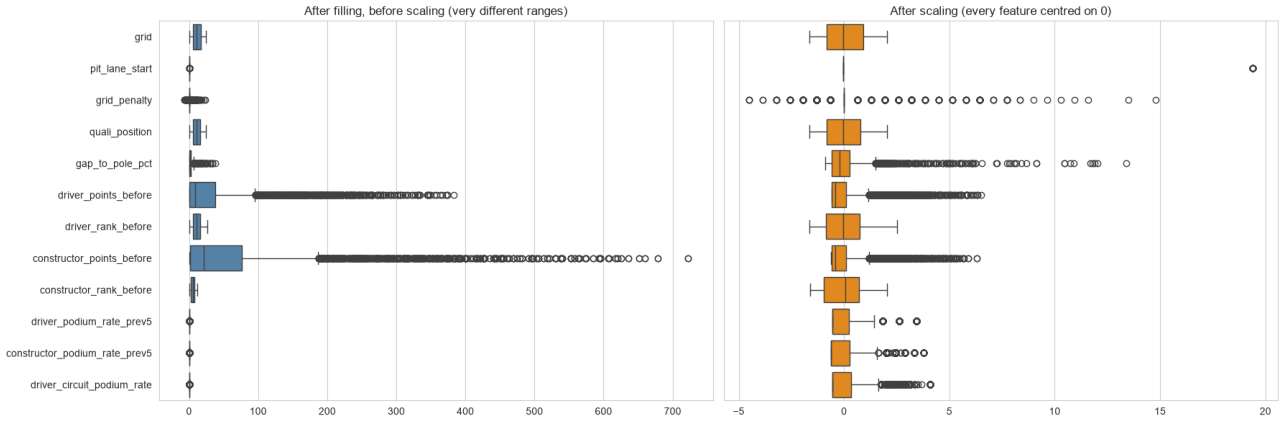


Figure 11: training features after filling (left) and after scaling (right).

6. Models and results

6.1 Metrics

Primary metric: PR-AUC (area under the precision-recall curve). It only looks at how well the model finds the rare podium rows, so it is the most honest metric for an imbalanced target. A random model scores about 0.14 (the podium share). **Secondary metric: ROC-AUC**, which measures how well podium rows are ranked above non-podium rows in general; it looks good easily when one class is rare, so it is second. We also report **F1, precision and recall** at one threshold. For each model the threshold is the one with the best F1 on the **validation** set. The test set is never used to choose anything.

6.2 The models

Model	How it works	Limitations
Rule: 1 / grid (reference, no ML)	score = 1 / grid; the threshold turns it into "podium if starting in the top 4"	uses only the grid
Logistic regression (baseline)	one weight per scaled feature, the weighted sum goes through a sigmoid to give a probability	only straight-line effects, no feature combinations
Random forest A (deep)	300 decision trees on random samples of rows and features, answers averaged; leaves of at least 3 rows; podium rows get more weight	deep trees can memorise the training rows
Random forest B (shallow)	same, but at most 6 levels deep and at least 20 rows per leaf	harder to explain than one weight per feature

Model	How it works	Limitations
Shallow neural network	1 hidden layer of 32 ReLU neurons, dropout 0.25, sigmoid output, Adam optimizer; early stopping on validation loss (patience 5)	hardest to explain; small data (6,400 rows) leaves little room to beat simpler models

6.3 Model comparison (all 5 metrics)

Model	Split	ROC-AUC	PR-AUC	F1	Precision	Recall	Threshold
Rule: 1 / grid	train	0.907	0.618	0.636	0.557	0.742	0.25
	validation	0.917	0.676	0.705	0.617	0.822	0.25
	test	0.914	0.647	0.665	0.583	0.775	0.25
Logistic regression	train	0.926	0.674	0.658	0.620	0.700	0.37
	validation	0.931	0.728	0.724	0.670	0.789	0.37
	test	0.935	0.708	0.668	0.638	0.701	0.37
Random forest A (deep)	train	0.991	0.945	0.816	0.689	0.999	0.40
	validation	0.936	0.735	0.721	0.632	0.839	0.40
	test	0.928	0.673	0.667	0.556	0.833	0.40
Random forest B (shallow)	train	0.946	0.731	0.683	0.660	0.709	0.76
	validation	0.939	0.744	0.729	0.686	0.778	0.76
	test	0.936	0.702	0.664	0.642	0.686	0.76
Shallow neural network	train	0.935	0.699	0.675	0.619	0.742	0.38
	validation	0.935	0.732	0.737	0.682	0.800	0.38
	test	0.932	0.693	0.661	0.600	0.735	0.38

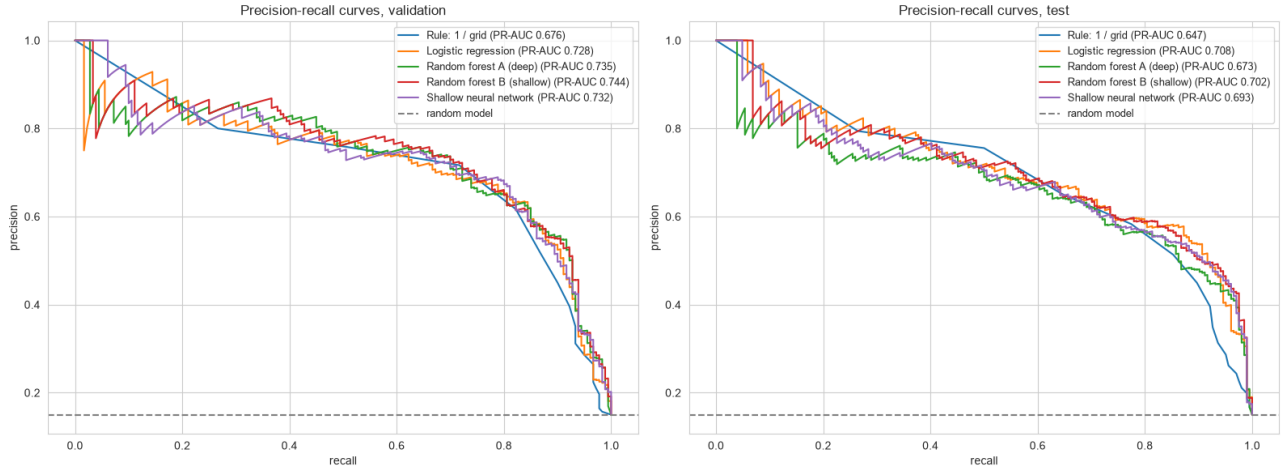


Figure 12: precision-recall curves on validation (left) and test (right). The dashed line is a random model.

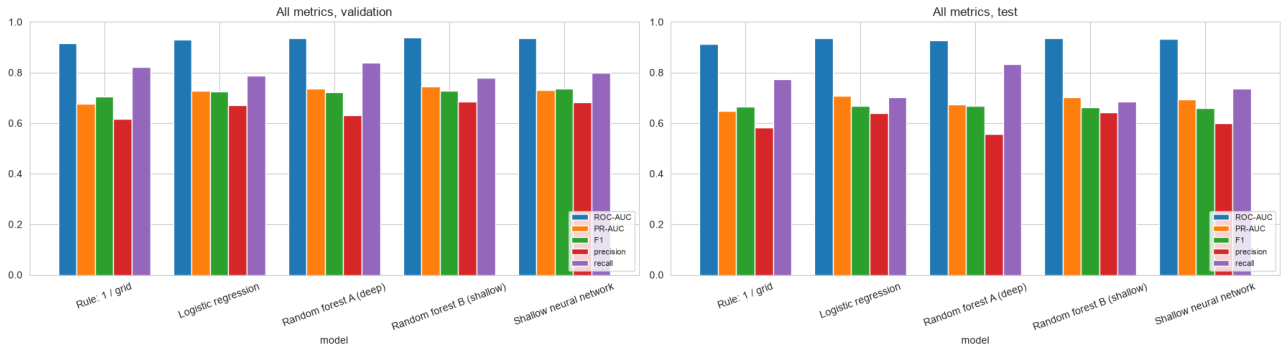


Figure 13: all five metrics per model on validation (left) and test (right).

Final model: random forest B, because it has the best validation PR-AUC (0.744) and the best validation ROC-AUC (0.939). The test set played no part in this choice.

- **Every model beats the grid rule** (validation PR-AUC 0.676), so form, standings and gap to pole add real information beyond the starting position.
- **Overfitting:** forest A scores 0.945 PR-AUC on train but 0.735 on validation, so it memorised the training rows. Limiting the depth (forest B) brings train and validation close (0.731 vs 0.744). This is why B beat A.
- **Test years:** all models drop a little on 2022 to 2024 (new car rules). Logistic regression is slightly ahead of forest B on test PR-AUC (0.708 vs 0.702), but we do not switch: choosing with the test set would make the test score dishonest. All models are within about 0.02, so the features matter more than the model.
- The forest's threshold is high (0.76) because it gives podium rows extra weight, which raises all its probabilities. The ranking of rows (PR-AUC, ROC-AUC) is not affected.

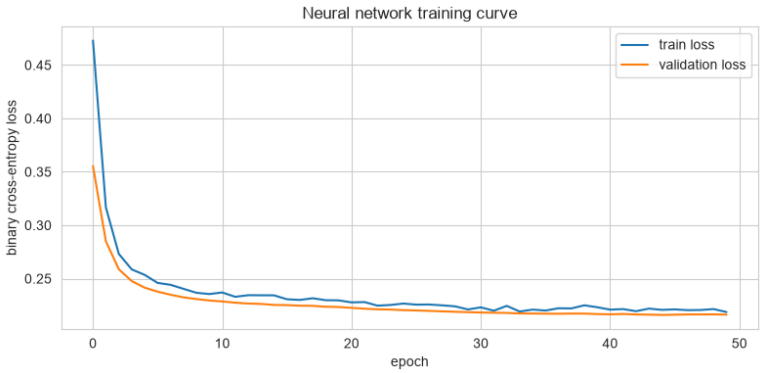


Figure 14: neural network loss per epoch. Train and validation loss fall together and level off, so the network did not overfit; early stopping ended training after 50 epochs.

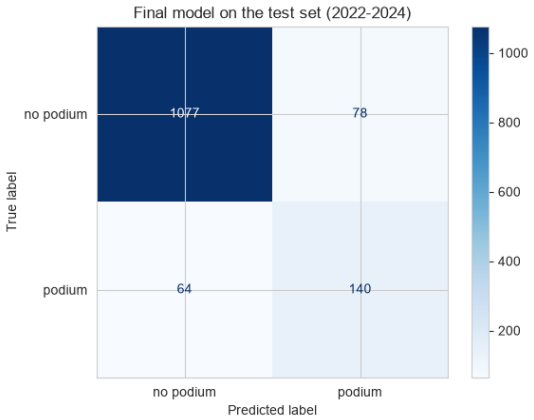


Figure 15: confusion matrix of the final model on the test set (validation threshold). It finds 140 of the 204 real podiums (recall 0.686), and 140 of its 218 podium predictions are correct (precision 0.642).

7. Experiments

We run the experiments with logistic regression: it trains in about a second, and unlike trees it reacts to scaling and filling, so preprocessing differences show up. Each run chooses its threshold on validation.

7.1 Feature-group ablation

We remove one group at a time, retrain, and compare with the full model. Because grid and qualifying (and standings and form) carry similar information, we also remove those pairs together.

Removed group	Validation PR-AUC	Change vs full	Validation F1	Test PR-AUC
none (all features)	0.728	0.000	0.725	0.708
grid	0.713	-0.015	0.718	0.702
qualifying	0.728	-0.000	0.723	0.709

Removed group	Validation PR-AUC	Change vs full	Validation F1	Test PR-AUC
standings	0.721	-0.007	0.726	0.703
form	0.734	+0.006	0.728	0.694
circuit history	0.724	-0.004	0.721	0.709
grid + qualifying (together)	0.630	-0.098	0.675	0.567
standings + form (together)	0.730	+0.002	0.734	0.675

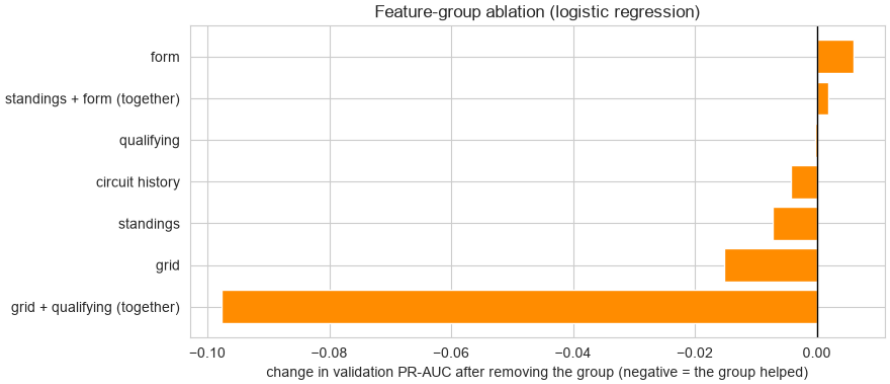


Figure 16: change in validation PR-AUC after removing each group (negative = the group helped).

- **Grid + qualifying together** is by far the most important information: without it, validation PR-AUC falls by about 0.10 and test PR-AUC from 0.708 to 0.567.
- Removed one at a time, grid (-0.015) and qualifying (about 0) look small, because they carry almost the same information (the grid equals the qualifying position in 72% of rows and is within one place in 89%).
- **Standings** (-0.007) and **circuit history** (-0.004) give small gains.
- **Form** is mixed: removing it raises validation PR-AUC a little (+0.006) but lowers test PR-AUC (0.708 to 0.694); removing standings and form together lowers test PR-AUC to 0.675. Changes this small on validation (about 180 podiums) are within noise, so we keep both groups, also because of the clear EDA evidence (Figure 9).

7.2 Preprocessing-order experiments

Experiment	Version	Validation PR-AUC	Validation F1	Test PR-AUC
1. Order	A: fill with median, then scale (used)	0.7277	0.7245	0.7083
1. Order	B: scale, then fill with median	0.7281	0.7245	0.7083
2. What it learns from	train rows only (used, correct)	0.7277	0.7245	0.7083
2. What it learns from	all rows before the split (leaky)	0.7275	0.7245	0.7082

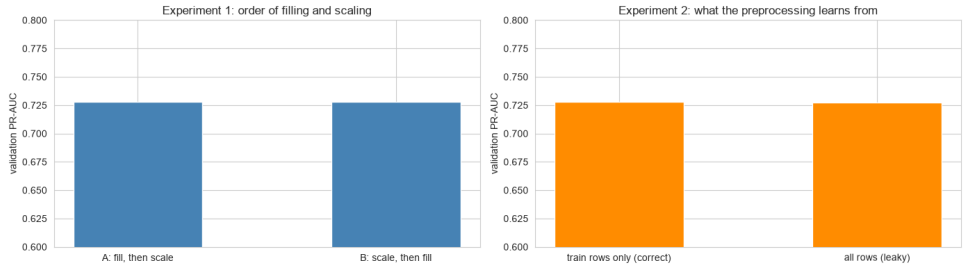


Figure 17: validation PR-AUC in the two preprocessing experiments.

- **Experiment 1:** the order makes almost no difference (0.7277 vs 0.7281), because few values are missing (24% in circuit history, below 8% in all other features). We keep fill-then-scale, because then the scaled data has exactly mean 0 and standard deviation 1, including the filled values.

- **Experiment 2:** learning the preprocessing on all rows changes what the scaler learns (the mean of constructor points goes from 64.3 to 76.9, because recent seasons give more points), but the scores barely change. It is still wrong in principle: at prediction time the future rows do not exist. On data where test looks more different from train, this leak can make scores look better than they are, so we keep the correct version.

8. Explainability (XAI)

We explain the final model (random forest B) on the test rows. **Important:** these methods show what the model **relies on** to make predictions. They do **not** show what **causes** a podium. For example, a good grid slot and a podium are both results of having a fast car: the model uses grid as a signal, but this does not prove that the grid slot itself causes the podium.

8.1 Global: permutation importance

We shuffle one feature at a time across the test rows (10 times each) and measure how much the test PR-AUC drops. A big drop means the model relies a lot on that feature.

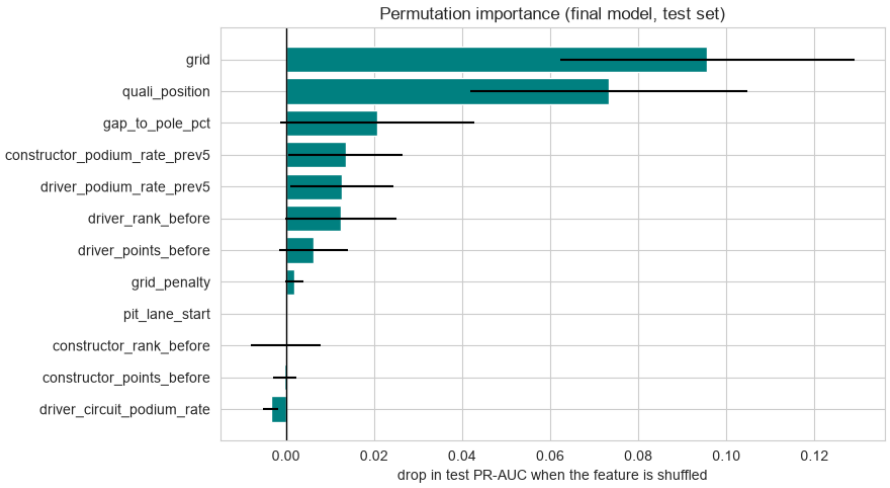


Figure 18: drop in test PR-AUC when each feature is shuffled (mean and standard deviation).

Shuffling **grid** lowers the PR-AUC the most (0.096), then **qualifying position** (0.073), gap to pole (0.021), team and driver form (0.014 and 0.013) and driver rank (0.013). The others are close to 0. This does not mean they are useless: when one is shuffled, a similar feature can take over (for example constructor rank and constructor form say almost the same thing). Small negative values are random noise.

8.2 Global: SHAP

SHAP splits each prediction into one push per feature, starting from the model's average prediction. The bar plot shows the average size of the push; the beeswarm shows one dot per test row, coloured by the feature value (red = high, blue = low).

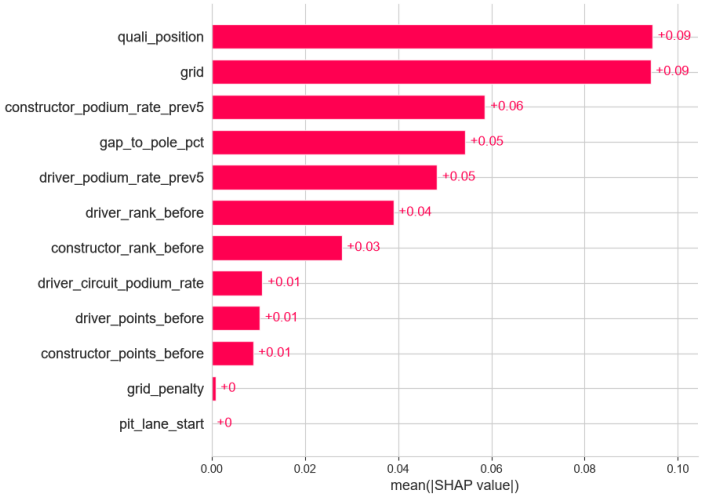


Figure 19: mean absolute SHAP value per feature.

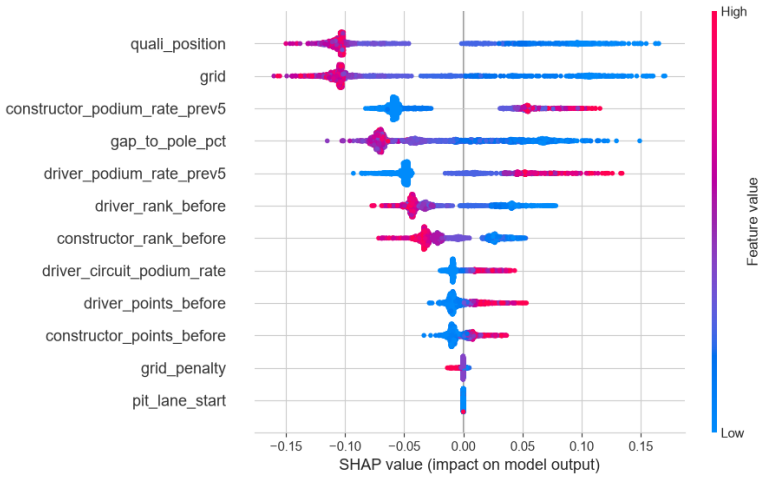


Figure 20: SHAP beeswarm for the test rows.

- The same features lead as in permutation importance: qualifying position and grid (about 0.09 each), then team form, gap to pole and driver form.
- **Direction:** a low (blue) qualifying position, grid and gap to pole push the probability up; high (red) driver and team form and points push it up; a low rank number (high in the championship) pushes it up. These directions match the EDA in Figure 9.
- Pit-lane start and grid penalty have almost no effect: they are rare and the grid value already includes the penalty.
- The SHAP base value is about 0.5 (not 0.14) because the forest gives podium rows extra weight.

8.3 Local: one prediction explained with SHAP and LIME

We picked a less obvious case: the first test row where a driver reached the podium after starting 6th or lower. **George Russell, 2022 Australian Grand Prix:** started 6th, finished 3rd. The model gave a probability of 0.467, below the threshold 0.76, so it predicted **no podium** and was wrong. We explain this prediction with two methods. The SHAP waterfall adds each feature's push to the average prediction. LIME makes many slightly changed copies of the row, asks the model about each one, and fits a simple straight-line model around it.

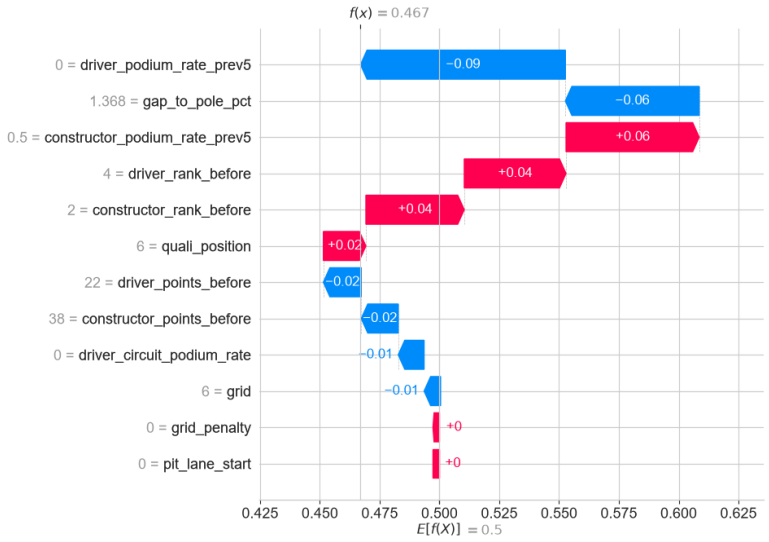


Figure 21: SHAP waterfall for Russell, 2022 Australia.

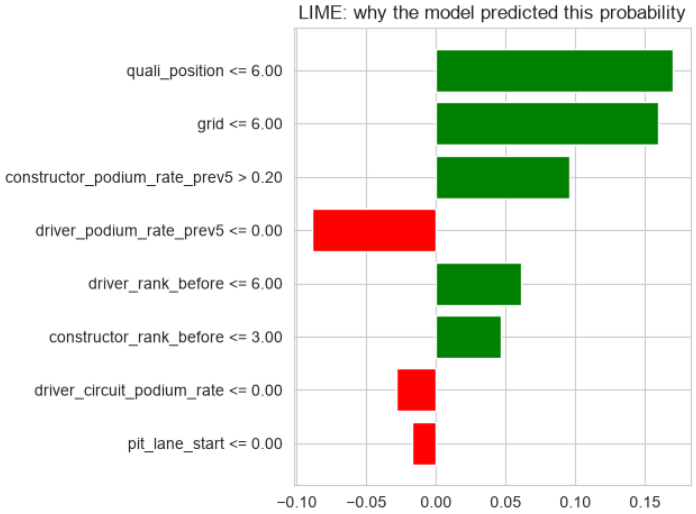


Figure 22: LIME explanation for the same row (green = towards podium, red = away).

- **Pushed down:** Russell had no podium in his previous 5 races (driver form = 0; SHAP -0.09, LIME -0.09). He had just moved from Williams to Mercedes, so his form came from a slower car. His gap to pole (1.37%) also pushed down in SHAP (-0.06).
- **Pushed up:** the team's recent form (Mercedes 0.5; SHAP +0.06, LIME +0.10) and the good championship ranks of the driver (4th) and team (2nd).
- **Grid and qualifying (P6):** LIME gives them the biggest positive push (+0.17 and +0.16), because P6 or better beats an average row. SHAP gives them only a small push, because it compares with the model's average prediction. The two methods answer slightly different questions but tell the same main story.
- **Lesson:** form follows the driver, so a driver who changes team carries outdated form for a few races.

9. Inference function

`predict_podium(record)` takes one driver-race as a dictionary of feature values and returns the podium probability and a yes/no prediction from the final model, with the threshold from validation. Missing features are filled with the training medians, exactly like in training.

Record	Missing features	Probability	Prediction
Complete: Russell, 2022 Australia	none	0.467	no podium
Partial: same race without qualifying data and driver history	quali_position, gap_to_pole_pct, driver_podium_rate_prev5, driver_circuit_podium_rate	0.335	no podium

Record	Missing features	Probability	Prediction
Front-runner: pole, championship leader, strong form	none	0.967	podium

The partial record still gets a prediction. Its probability drops (0.467 to 0.335) because the median qualifying position (11) is worse than Russell's real one (6): missing values can change a prediction a lot.

10. Limitations

- **Qualifying-coverage drift:** qualifying data is incomplete before 2003, so the model only learns from 2003 on and does not cover older eras.
- **Era sensitivity:** car rules change (hybrid engines in 2014, new cars in 2022) and the points system changed in 2010. Scores drop a little on 2022 to 2024, and a model trained on old seasons may not fit a new era.
- **Team changes:** form follows the driver, so a driver who moves to a better or worse team carries outdated form for a few races (the Russell example).
- **Rare class:** podiums are about 14% of rows. F1, precision and recall depend on the validation threshold, and validation has only about 180 podiums, so small differences between models are within noise.
- **Missing values:** a rookie or a driver without a qualifying time gets training medians, which can move the prediction a lot.
- **Data questions:** home country is approximated from nationality, and the mechanical-retirement list was made by hand (unclear statuses excluded).
- **Not causal:** permutation importance, SHAP and LIME describe the model, not the causes of race results.

Data source

Formula 1 World Championship (1950 to 2024), Kaggle / Ergast dataset: kaggle.com/datasets/rohanrao/formula-1-world-championship-1950-2020. Libraries: pandas, numpy, matplotlib, seaborn, scikit-learn, TensorFlow/Keras, SHAP, LIME.